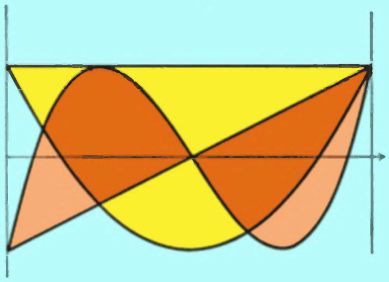
Numerical Methods for Scientists and Engineers



R.W. Hamming

Second Edition

Numerical Methods for Scientists and Engineers

SECOND EDITION

R. W. HAMMING

Adjunct Professor of Computer Science,
Naval Postgraduate School

Dover Publications, Inc., New York

This Dover edition, first published in 1986, is an unabridged republication of the second edition (1973) of the work first published by McGraw-Hill, Inc., New York, in 1962.

Manufactured in the United States of America
Dover Publications, Inc., 31 East 2nd Street, Mineola, N.Y. 11501

Library of Congress Cataloging-in-Publication Data

Hamming, R. W. (Richard Wesley), 1915-
Numerical methods for scientists and engineers.

Reprint. Originally published: New York: McGraw-Hill, 1973.
Bibliography: p.
Includes index.
1. Numerical analysis—Data processing. I. Title.
[QA297.H28 1987] 519.4 86-16226
ISBN 0-486-65241-6

THE PURPOSE OF COMPUTING
IS INSIGHT, NOT NUMBERS

To study, and when the occasion arises to put what one has learned into practice—is that not deeply satisfying?
Confucius, Analects 1.1.1

CONTENTS

II Polynomial Approximation—Classical Theory

III Fourier Approximation—Modern Theory

IV Exponential Approximation

V Miscellaneous

* Starred sections may be omitted.

PREFACE

There has been much progress in the 10 years since the first edition was written, but of the many books that have appeared on the topic none has put the emphasis on the frequency approach and its use in the solution of problems. For these reasons, a second edition seems necessary.

The material has been extensively rearranged, rewritten, and added to, so that in some respects it is a new book; however, the main aims, style, and motto have not changed.

As always, the author is greatly indebted to others for much that is in the book. Most important are his management and colleagues at the Bell Telephone Laboratories. Professor Roger Pinkham has over the years been a constant source of stimulation and inspiration. It would take a list of at least 100 names to thank all who have contributed to some extent, and at the top of this list would be M. P. Epstein. My thanks also go to all the unmentioned people on the list and to A. Ralston for many helpful suggestions. Thanks also to Mrs. Jeannie Waddel for typing and helping to organize the manuscript.

R. W. HAMMING

PART I

Fundamentals and Algorithms

1

AN ESSAY ON NUMERICAL METHODS

1.1 THE FIVE MAIN IDEAS

Numerical methods use numbers to simulate mathematical processes, which in turn usually simulate real-world situations. This implies that there is a *purpose* behind the computing. To cite the motto of the book, The Purpose of Computing Is Insight, Not Numbers. This motto is often thought to mean that the numbers from a computing machine should be read and used, but there is much more to the motto. The choice of the particular formula, or algorithm, influences not only the computing but also how we are to understand the results when they are obtained. The way the computing progresses, the number of iterations it requires, or the spacing used by a formula, often sheds light on the problem. Finally, the same computation can be viewed as coming from different models, and these different views often shed further light on the problem. *Thus computing is, or at least should be, intimately bound up with both the source of the problem and the use that is going to be made of the answers—it is not a step to be taken in isolation from reality.*

Much of the knowledge necessary to meet this goal comes from the field of application and therefore lies outside a general treatment of numerical methods. About all that can be done is to supply a rich assortment of methods and to

comment on their relevance in general situations. This art of connecting the specific problem with the computing is important, but it is best taught in connection with a field of application.

The second main idea is a consequence of the first. If the purpose of computing is insight, not numbers, as the motto states, then it is necessary to *study families and to relate one family to another when possible*, and to avoid isolated formulas and isolated algorithms. In this way a sensible choice can be made among the alternate ways of doing the problem, and once the computation is done, alternate ways of viewing the results can be developed. Thus, hopefully, the insight can arise. For these reasons we tend to concentrate on systematic methods for finding formulas and avoid the isolated, cute result. It is somewhat more difficult to systematize algorithms, but a unifying principle has been found.

This is perhaps the place to discuss some of the differences between numerical methods and numerical analysis (as judged by the corresponding textbooks). Numerical analysis seems to be the study in depth of a few, somewhat arbitrarily selected, topics and is carried out in a formal mathematical way devoid of relevance to the real world. Numerical methods, on the other hand, try to meet the need for methods to cope with the potentially infinite variety of problems that can arise in practice. The methods given are generally chosen for their wide applicability in creating formulas and algorithms as well as for the particular result being found at that point.

The third major idea is *roundoff error*. This effect arises from the finite nature of the computing machine which can only deal with finitely represented numbers. But the machine is used to simulate the mathematician's number system which uses infinitely long representations. In the machine the fraction $\frac{1}{3}$ becomes the terminated decimal 0.333...3 with the obvious roundoff effect. At first this approximation does not seem to be very severe since usually a minimum of eight decimal places are carried at every step, but the incredible number of arithmetic operations that can occur in a problem lasting only a few seconds is the reason that roundoff plays an important role. *The greatest loss of significance in the numbers occurs when two numbers of about the same size are subtracted so that most of the leading digits cancel out, and unless care is taken in advance, this can happen almost any place in a long computation.*

Most books on computing stress the estimation of roundoff, especially the bounding of roundoff, but we shall concentrate *on the avoidance of roundoff*. It seems better to avoid roundoff than to estimate what did not have to occur if common sense and a few simple rules had been followed *before* the problem was put on the machine.

The fourth main idea is again connected with the finite nature of the machine, namely that many of the processes of mathematics, such as differentiation and in-

tegration, imply the use of a limit which is an infinite process. The machine has finite speed and can only do a finite number of operations in a finite length of time. This effect gives rise to the *truncation error* of a process.

We shall generally first give an exact expression for the truncation error and deduce from it various bounds. A moment's thought should reveal that if we had an exact expression, then it would be practically useless because to know the exact error is to know the exact answer. However, the exact-error expression is very useful in studying families of formulas, and it provides a starting point for a variety of error bounds.

The fifth main idea is *feedback*, which means, as its name implies, that numbers produced at one stage are fed back into the computer to be processed again and again; the program has a loop which uses the output of one cycle as the input for the next cycle. This feedback situation is very common in computing, as it is a very powerful tool for solving many problems.

Feedback leads immediately to the associated idea of *stability* of the feedback loop—will a small error grow or decay through the successive iterations? The answer may be given loosely in two equivalent ways: first, if the feedback of the error is too strong and is in the direction to eliminate the error (technically, negative feedback), then the system will break into an oscillation that grows with time; second and equivalently, if the feedback is delayed too long, the same thing will happen.

A simple example that illustrates feedback instability is the common home shower. Typically the shower begins with the water being too cold, and the user turns up the hot water to get the temperature he wants. If the adjustment is too strong (he turns the knob too far), he will soon find that the shower is too hot, whereupon he rapidly turns back to cold and soon finds it is too cold. If the reactions are too strong, or alternately the total system (pipes, valve, and human) is too slow, there will result a "hunting" that grows more and more violent as time goes on. Another familiar example is the beginning automobile driver who overreacts while steering and swings from side to side of the street. This same kind of behavior can happen for the same reasons in feedback computing situations, and therefore the stability of a feedback system needs to be studied before it is put on the computer.

1.2 SECOND-LEVEL IDEAS

Below the main ideas in Sec. 1.1 are about 50 second-level ideas which are involved in both theoretical and practical work. Some of these are now discussed.

At the foundation of all numerical computing are the actual numbers

themselves. The floating-point number system used in most scientific and engineering computations is significantly different from the mathematician's usual number system. The floating-point numbers are not equally spaced, and the numbers do not occur with equal frequency. For example, it is well known that a table of physical constants will have about 60 percent of the numbers with a leading digit of 1, 2, or 3, and the other digits—4, 5, 6, 7, 8, and 9—comprise only 40 percent.

Although this number system lies at the foundation of most of computing, it is rarely investigated with any care. People tend to start computing, and only after having frequent trouble do they begin to look at the system that is causing it.

Immediately above the number system is the apparently simple matter of evaluating functions accurately. Again people tend to think that they know how to do it, and it takes a lot of painful experience to teach them to examine the processes they use before putting them on a computer.

These two mundane, pedestrian topics need to be examined with the care they deserve *before* going on to more advanced matters; otherwise they will continually intrude in later developments.

Perhaps the simplest problem in computing is that of finding the zeros of a function. In the evaluation of a function near a zero there is almost exact cancellation of the positive and negative parts, and the two topics we just discussed, roundoff of the numbers and function evaluation, are basic, since if we do not compute the function accurately, there can be little meaning to the zeros we find. Because of the discrete structure of the computer's number system it is very unlikely that there will be a number x which will make the function $y = f(x)$ exactly zero. Instead, we generally find a small interval in which the function changes sign. The size of the interval we can use is related to the size of the argument x, since for large x the number system has a coarse spacing and for x small (in size) it has a fine spacing. This is one of the reasons that the idea of the *relative error*

$$\text{Relative error} = \left| \frac{\text{true} - \text{calculated}}{\text{true}} \right|$$

plays such a leading role in scientific and engineering computations. Classical mathematics uses the *absolute error*

$$\text{Absolute error} = |\text{true} - \text{calculated}|$$

most of the time, and it requires a positive effort to unlearn the habits acquired in the conventional mathematics courses. The relative error has trouble near places where the true value is approximately zero, and in such cases it is customary to use as the denominator

$$\max\{|x|, |f(x)|\}$$

where $f(x)$ is the function computed at x.

The problem of finding the complex zeros of an analytic function occurs so often in practice that it cannot be ignored in a course on numerical methods, though it is almost never mentioned in numerical analysis. A simple method resembling one used to find the real zeros is very effective in practice.

In the special case of finding all the zeros of a polynomial the fact that the number of zeros (as well as other special characteristics) is known in advance makes the problem easier than for the general analytic function. One of the best methods for finding them is an adaptation of the usual Newton's method for finding real zeros, and this discussion is used to extend, as well as to analyse further, Newton's method. It is only in situations in which a careful analysis can be made that Newton's method is useful in practice; otherwise its well-known defects outweigh its virtues.

What makes the problem of finding the zeros of a polynomial especially important, besides its frequency, is the use made of the zeros found. The method is a good example of the difference between the mathematical approach and the engineering approach. The first merely tries to find some numbers which make the function close to zero, while the second recognizes that a pair of "close" zeros will give rise to severe roundoff troubles when used at a later stage. In isolation the problem of finding the zeros is not a realistic problem since the zeros are to be used, not merely admired in a vacuum. Thus what is wanted in most practice is the finding of the multiple zeros as multiple zeros, not as close, separate ones. Similarly, zeros which are purely imaginary are to be preferred to ones with a small real part and a large imaginary part, *provided* the difference can reasonably be attributed to uncertainties in the underlying model.

Another standard algorithmic problem both in mathematics and in the use of computation to solve problems is the solution of simultaneous linear equations. Unfortunately much of what is commonly taught is usually not relevant to the problem as it occurs in practice; nor is any completely satisfactory method of solution known at present. Because the solution of simultaneous linear equations is so often a standard library package supplied by the computing center and because the corresponding description is so often misleading, it is necessary to discuss the limitations (and often the plain foolishness) of the method used by the package. Thus it is necessary to examine carefully the obvious flaws and limitations, rather than pretending they do not exist.

The various algorithms for finding zeros, solving simultaneous linear equations, and inverting matrices are the classic algorithms of numerical analysis. Each is usually developed as a special trick, with no effort to show any underlying principles. The idea of an *invariant algorithm* provides one common idea linking, or excluding, various methods. An invariant algorithm is one that in a very real sense attacks the problem rather than the particular representation supplied to the

computer. The idea of an invariant algorithm is actually fairly simple and obvious once understood. In many kinds of problems there are one or more classes of transformations that will transform one representation of the equations into another of the same form. For example, given a polynomial

$$P(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_0 = 0$$

the transformation of multiplying the equation by any nonzero constant does not really change the problem. Similarly, when $a_0 \neq 0$, replacing x by $1/x$ while also multiplying the equation by x^n merely reverses coefficients. These transformations form a group (provided we recognize the finite limitations of computing), and it is natural to ask for algorithms that are invariant with respect to this group, where invariant means that if the problem is transformed to some equivalent form, then the algorithm uses, at all stages, the equivalent numbers (within roundoff, of course). In a sense the invariance is like dimensional analysis—the scaling of the problem should scale the algorithm in exactly the same way. It is more than dimensional analysis since, as in the example of the polynomial, some of the transformations to be used in the problem may involve more than simple scaling.

It is surprising how many common algorithms do not satisfy this criterion. The principle does more than merely reject some methods; it also, like dimensional analysis, points the way to proper ones by indicating possible forms that might be tried.

1.3 THE FINITE DIFFERENCE CALCULUS

After examining the simpler algorithms, it is necessary to develop more general tools if we are to go further. The *finite difference calculus* provides both the notation and the framework of ideas for many computations. The finite difference calculus is analogous to the usual infinitesimal calculus. There are the difference calculus, the summation calculus, and difference equations. Each has slight variations from the corresponding infinitesimal calculus because instead of going to the limit, the finite calculus stops at a fixed step size. This reveals why the finite calculus is relevant to many applications of computing: in a sense it *undoes* the limiting process of the usual calculus. It should be evident that if a limit process cannot be undone, then there is a very real question as to the soundness of the original derivation, because it is usually based on constructing a believable finite approximation and then going to the limit.

The finite difference calculus provides a tool for estimating the roundoff effects that appear in a table of numbers *regardless* of how the table was computed. This tool is of broad and useful application because instead of carefully studying

each particular computation, we can apply this general method without regard to the details of the computation. Of course, such a general method is not as powerful as special methods hand-tailored to the problem, but for much of computation it saves both trouble and time.

The summation calculus provides a natural tool for approaching the very common (and often neglected) problem of the summation of infinite series, which is the simplest of the limiting processes (since the index n of the number of terms taken runs through the integers only).

The solution of finite difference equations is analogous to the solution of differential equations, especially the very common case of linear difference equations with constant coefficients, which is a valuable tool for the study of feedback loops and their stability. Thus finite difference equations have both a practical and a theoretical value in computing.

1.4 ON FINDING FORMULAS

Once past the easier algorithms and tools for doing simple things in computing, it is natural to attack one of the central problems of numerical methods, namely, the approximation of infinite operations (operators) by finite methods. Interpolation is the simplest case. In interpolation we are given some samples of the function, say, $y(-1)$, $y(0)$, and $y(1)$, and we are asked to *guess* at the missing values—to read between the lines of a table. While it is true that because of the finite nature of the number system used there are only a finite number of values to be found, nevertheless this number is so high that it might as well be infinite. Thus interpolation is an infinite operator to be approximated.

There is no sense to the question of interpolation unless some additional assumptions are made. The *classical assumption* is that given $n + 1$ samples of the function, these samples determine a unique polynomial of degree n, and this polynomial is to be used to give the interpolated values. With the above data consisting of three points, the quadratic through these points is

$$P(x) = \frac{x(x-1)}{2} y(-1) + (1 - x^2)y(0) + \frac{x(x+1)}{2} y(1)$$

We are to use this polynomial $P(x)$ as if it were the function. This method is known as the *exact matching* of the function to the data.

The error of this interpolation can be expressed as the $(n + 1)$st derivative (of the original function) evaluated at some generally unknown point 0 in the interval. Unfortunately in practice it is rare to have any idea of the size of this derivative.

For samples of the function we may use not only function values $y(x)$ but also values of the derivatives $y'(x)$, $y''(x)$, etc., at various points. For example, the cubic exactly matching the data $y(0)$, $y(1)$, $y'(0)$, and $y'(1)$ is

$$P(x) = (1 - 3x^2 + 2x^3)y(0) + (3x^2 - 2x^3)y(1) + (x - 2x^2 + x^3)y'(0) + (x^3 - x^2)y'(1)$$

It is important to use analytically found derivatives when possible. Then we can usually get a higher order of approximation at little extra cost since generally once the function values are computed, the derivatives are relatively easy to compute. No new radicals, logs, exponentials, etc., arise, and these are the time-consuming parts of most function evaluation. Of course a sine goes into a cosine when differentiated, but this is about the only new term needed for the higher derivatives. Even the higher transcendental functions, like the Bessel functions, satisfy a second-order linear differential equation, and once both the function and the first derivative are found, the higher derivatives can be computed from the differential equation and its derivatives (which are easy to compute). Thus we shall emphasize the use of derivatives as well as function values for our samples.

Although a wide variety of function and derivative values may be used to determine the interpolating polynomial, there are some sets, rather naturally occurring, for which $n + 1$ data samples do not determine an nth-degree polynomial. Perhaps the best example is the data $y(-1)$, $y(0)$, $y(1)$, $y''(-1)$, $y''(0)$, and $y''(1)$ which do not determine a fifth-degree polynomial—the positions and accelerations at three equally spaced points do not determine a quintic in general.

The classic method for finding formulas for other infinite operators, such as integration and differentiation, is to use the interpolating polynomial as if it were the function and then to apply the infinite operator to the polynomial. For example, if we wish to find the integral of a function from -1 to $+1$, given the values $y(-1)$, $y(0)$, and $y(1)$, we find the interpolating quadratic as above and integrate it to get the classical Simpson's formula:

$$\int_{-1}^{1} y(x)\,dx = \tfrac{1}{3}y(-1) + \tfrac{4}{3}y(0) + \tfrac{1}{3}y(1)$$

This process is called *analytic substitution*; in place of the function we could not handle we take some samples, exactly match a polynomial to the data, and finally analytically operate on this polynomial. This is the classical method for finding formulas. It is a two-step method: find the interpolating function and then apply the operator to this function.

There is another direct method that is *almost* equivalent to the analytic-substitution method. In this method we make the formula true for a sequence of

functions $y(x) = 1, x, x^2, x^3, \ldots, x^m$. For example, to derive Simpson's formula by this method we assume the form

$$\int_{-1}^{1} y(x)\,dx = a_{-1}y(-1) + a_0\,y(0) + a_1 y(1)$$

and substitute the sequence of functions 1, x, and x^2. The three resulting equations determine the three unknown coefficients a_i, and the resulting formula is exactly the same (in this case). The two methods differ in the case where there is no interpolating polynomial; it may be that there is a formula even if there is no interpolating polynomial. For example, we have the formula

$$\int_{-1}^{1} y(x)\,dx = \tfrac{1}{21}[5y(-1) + 32y(0) + 5y(1)] - \tfrac{1}{315}[y''(-1) - 32y''(0) + y''(1)]$$

which is exact for sixth-degree polynomials when, as we have noted above, there is in general no interpolating polynomial of fifth degree.

It would seem as if the two methods were equivalent, for if there were an interpolating polynomial, then the formula would surely be true for the corresponding powers of x; and conversely, if it were true for the individual powers, then it would be true for any linear combination, namely a polynomial. The difference lies in the words *if there is an interpolating polynomial, then* .... It can happen that the two-stage process fails on the first step, but the one-step direct method will work.

There are two main advantages of the direct method. First the derivations are much easier, and second the direct method provides a basis for extensive generalizations. The importance of this method is hard to overestimate. It means that we can find a very wide range of formulas, all within a common framework of ideas and methods, and that we will therefore be able to compare one formula to another and then decide which one to use. Perhaps more important, it means that we can decide the kind of formula we want and then with this single method and its generalizations find almost any formula we want—we can fit the formula to the problem rather than fit the problem to the formula. Thus we can try to achieve the insight that is the main goal of the book, as stated in our motto, The Purpose of Computing Is Insight, Not Numbers.

With a general method for finding polynomial approximation formulas it is necessary to have a corresponding method for finding the error of the formula. The general method of finding the error is somewhat difficult to understand the first time. It is based on the use of a Taylor series with the integral remainder, and by substituting this into the formula for which we want the error (and manipulating the results a bit) we get the desired error formula. Once we have the exact-error term, it can be transformed in various ways to get suitable practical-error estimates.

The method for finding the truncation error term for polynomial approximation unfortunately gives it in the form of a derivative, much as the interpolation method did. This, as noted before, is unfortunate because the high-order derivatives are seldom available.

This brings up a central dilemma. Should one use a high-order formula (error term has a high-order derivative) or use the repetition of a low-order formula—the composite formula? The answer is simple in principle. It depends on the size of the high-order derivative as compared to the other lower-order derivative. This is, of course, almost no answer at all, because we seldom can decide which is better, and the basis of the choice depends on the location in the complex plane of the singularities of the function being integrated—something we seldom know.

1.5 CLASSICAL NUMERICAL ANALYSIS

Much of classical numerical analysis, as we have indicated, is based on polynomial approximation for the infinite operations of differentiation, integration, and interpolation. The polynomial approximation is also used in the numerical integration of ordinary differential equations. The most widely used methods for this are the predictor-corrector methods. A polynomial is fitted to some of the data at past points and is used to extrapolate to the next point—to *predict*. The predicted value is used in the differential equation to get the predicted slope. This slope along with past data is used to find another polynomial which produces the *corrected value*, and the corrected value of the slope is found. If the predicted and corrected values are sufficiently close, then the step is accepted as accurate enough, and if not, the step size of integration may be halved (doubled if the two values are too close).

There are so many possible predictor-corrector methods of the same order of accuracy that it is necessary to have a general theory to compare the various formulas within a common framework. Otherwise chaos and prejudice would reign.

So far we have discussed the exact-matching interpolating polynomial, and this is the more usual method. There are other methods, more or less classical, for the selection of the approximating polynomial. One method is the minimum sum of squares of the residuals between the formula and the data given. Another more modern method picks the polynomial with the minimum maximum error—the minimax, or Chebyshev, approximation. Still other criteria could be used if desired, though the labor of finding the polynomial may be fairly high in some cases.

1.6 MODERN NUMERICAL METHODS— FOURIER APPROXIMATION

The difficulty with polynomial approximation in practice is that it is in the nature of polynomials to "wiggle" and to go to infinity for large absolute values of the argument x. Physically occurring functions tend to wiggle much less than polynomials and to remain bounded for large values of the argument. Thus polynomials are a poor basis for approximation, even though they are easy to compute and to think about. The fact that the Weierstrass approximation theorem states that any continuous function can be uniformly approximated in a closed interval by a polynomial is irrelevant for two reasons. First, the degree of the Weierstrass polynomial is generally very high for even a low degree of approximation; second, we are not finding the polynomial the way the theorem states it can be found. Indeed, it is "well known"[1] that for the simple function $y(x) = 1/(1 + x^2)$ in the interval $|x| \leq 3.63\ldots$ the sequence of polynomials that exactly matches the function at a set of equally spaced points does not approach the function uniformly as the number of points increases indefinitely—the function and the polynomial differ by arbitrarily large amounts, and the sequence of approximating polynomials fails to converge.

Since polynomials are rather poor functions to use for approximating many functions that occur in practice, it is natural to look for other sets of functions. Among the many sets that are known to be complete (meaning that they can approximate any continuous function in a closed interval) the functions $\sin nx$ and $\cos nx$ $(n = 0, 1, \ldots)$ have been the most studied and are the most useful. Approximation in terms of them is usually called Fourier approximation because J. B. J. Fourier (1768–1830) used them extensively in his work.

In the simplest case of approximating a function in an interval we are given a periodic function of period, say 2π, and are asked to approximate the function $y(x)$ by a form

$$y(x) = \frac{a_0}{2} + \sum_{k=1}^{\infty} (a_k \cos kx + b_k \sin kx)$$

It is easy to show that since the functions are orthogonal, that is,

$$\int_0^{2\pi} \cos kx \cos mx\, dx = \begin{cases} 0 & k \neq m \\ \pi & k = m \neq 0 \\ 2\pi & k = m = 0 \end{cases}$$

$$\int_0^{2\pi} \sin kx \cos mx\, dx = 0$$

$$\int_0^{2\pi} \sin kx \sin mx\, dx = \begin{cases} 0 & k \neq m \\ \pi & k = m \neq 0 \end{cases}$$

[1] Meaning that it can be found in the literature.

the coefficients in the expansion are given by

$$a_k = \frac{1}{\pi}\int_0^{2\pi} y(x)\cos kx\,dx \qquad b_k = \frac{1}{\pi}\int_0^{2\pi} y(x)\sin kx\,dx$$

The Fourier functions have a number of interesting properties beyond merely remaining bounded for all values. The error of an approximation that uses only a finite number of terms can be expressed in terms of the function rather than, as in the polynomial case, some high-order derivative. Furthermore, the rate of convergence, that is, how fast the finite series approaches the function as we take more and more terms, can be estimated easily from the discontinuities of the function and its derivatives.

Perhaps most important is the simple fact that the effect of taking equally spaced samples of the continuous function can be easily understood. The higher frequencies (speeds of rotation) appear as if they were lower frequencies. This effect, called *aliasing* because one frequency goes under the name of another, is a familiar phenomenon to the watchers of TV and movie westerns. As the stage coach starts up, the wheels start going faster and faster, but then they gradually slow down, stop, go backwards, slow down, stop, go forward, etc. This effect is due *solely* to the sampling the picture makes of the real scene. Figures 1.6.1 and 1.6.2 should make the effect clear. Once we know the sampling rate, we know exactly what frequencies will go into what frequencies. The highest frequency that is correct is called the *Nyquist*, or *folding*, *frequency*. In the polynomial situation we have no such simple understanding of the effect of sampling.

It might appear that to calculate all the coefficients of a Fourier expansion would involve a great deal of computing, but the recently discovered fast Fourier transform (FFT) method requires about $N \log N$ operations to fit N data points. This discovery has greatly increased the importance of Fourier approximation.

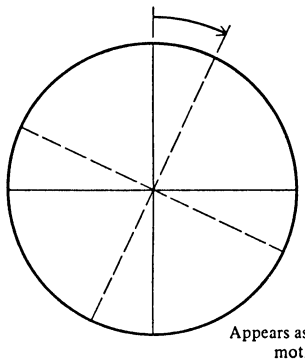


FIGURE 1.6.1

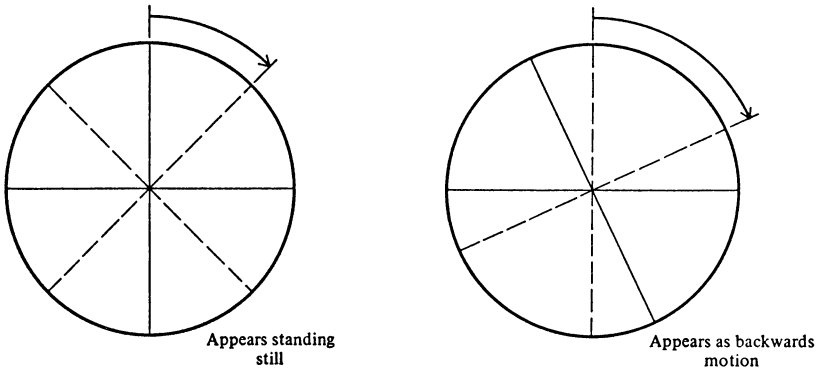


FIGURE 1.6.2

Most of our functions are not periodic, and so the Fourier series is of limited importance. For the general nonperiodic function there is a corresponding Fourier integral

$$f(t) = \int_{-\infty}^{\infty} F(\sigma)e^{2\pi i\sigma t}\,d\sigma$$

$$F(\sigma) = \int_{-\infty}^{\infty} f(t)e^{-2\pi i\sigma t}\,dt$$

The two functions $f(t)$ and the *transform* $F(\sigma)$ have the same information: one describes the function in the space of the variable t, while the other describes it in the space of frequencies σ. These two equivalent views are part of the reason that the Fourier integral gives such a useful approach to many problems.

When we resort to using a finite number of sample points, we again face aliasing, exactly the same effect as before. The folding frequency now provides a more natural barrier and leads to the concept of a "band-limited function," meaning that the frequencies in the function are confined to a band.

The idea of a band-limited function is mirrored very closely in real problems. A hi-fi system handles all the frequencies in a band and cuts off above and below rather sharply; the better the hi-fi system, the wider the band. The idea of a band of frequencies applies to most information transmission systems, servomechanisms, and feedback control situations. This means, among other things, that from the physics of the problem we can estimate the spacing we will need for our samples, and vice versa, that from the spacing we can estimate the frequency content of the solution.

With the aid of the Fourier integral, which closely parallels the Fourier series with its nice mathematical properties, we can show the effect of taking a finite slice of a function from a potentially infinitely long function. For example, the light from a pulsar or Cepheid variable star shines for many years; yet we observe it for a night or two and from that limited record try to estimate what the star is doing. It is often important to know the effect of the length of the observation on what we can hope to learn about the star. Similarly in other control problems, the length of the observation affects what we can see, and the Fourier integral enables us to understand this limitation.

Using this new tool of Fourier approximation, we can then look back at the polynomial approximation methods and see them in a new way. Once this new way becomes familiar, we can understand much more clearly what we were doing before. Exactly the same computations can be viewed in a new, more revealing light. This is especially true for the communications and control problems that tend to dominate our technological age.

Once this new way of looking at computing becomes familiar, it is natural to begin designing formulas to meet new criteria. In place of the discrete set of integers that were the exponents of the variable x in polynomial approximation, we now have a continuous band of frequencies to use and examine. We can pick the formula that minimizes some property of this continuous error curve (in the frequency space to be sure). One popular method is to make the error curve have a minimax (Chebyshev) property.

This new approach is relevant to many design problems. For example, in designing a simulator for humans to use while training for airplane or space travel, we are interested in building the simulator so that it "feels right" to the human who is being trained. This to a first approximation means that the Fourier transform (more generally, the Laplace transform) of the simulator should be close to the transform of the real thing. It is less important that the simulator and real vehicle go exactly the same place than it is to "feel right." Thus we are no longer to judge the method of integrating a system of differential equations that describe the simulator by how well the solutions agree, but rather by how well the transforms agree—which is not the same thing! This new method of design is known as *the method of zeros and poles*, and unfortunately it involves classical network theory.

The role of digital communications systems is steadily increasing; more and more we are sampling the continuous analog signal that occurs naturally in the real world and converting it to a sequence of discrete digits. Thus we face not only sampling but *quantization effects* of the digitizing of the analog signal. This effect is like that of roundoff in many ways, but it is usually much more severe, and because it occurs at the beginning, it again limits us to what we can

hope to see of the original underlying physical phenomena. Without a good understanding of these limitations we are not likely to understand what the numbers coming out of the computer mean and do not mean.

This leads gradually to the design of digital filters, which filter digital signals much as the old analog filters were used in processing analog signals, with radio and television being a couple of familiar examples. The differences from the continuous signals are significant in the digital case, and it is the digital case that digital computers must use.

1.7 OTHER CLASSES OF FUNCTIONS USED IN APPROXIMATIONS

After polynomial and Fourier approximations, the exponential set of functions is most used. The three sets, polynomials, Fourier, and exponentials (together with combinations of the three), are *invariant* under a translation of the origin. This is an important property because in many situations there is no natural origin, and without this property the choice of the origin would affect the answer. For these three classes the answer is independent of the origin, though the particular set of coefficients used in the approximation will differ as the origin is chosen differently.

For the exponential functions there is the Laplace transform corresponding to the Fourier transform, but it has much more difficult properties from the computing point of view.

1.8 MISCELLANEOUS

When a problem has a singularity, as many practical problems do (often because of the mathematical idealization), then the structure of the singularity indicates the class of approximating functions to use, and the position gives the natural origin. The methods used in this case are similar to those of the three usual cases.

Sometimes the problem has a natural set of functions to use, and again the methods we have developed can be applied, though the details may get a bit messy in many cases.

Optimization occurs frequently in practice, and the person practicing numerical methods needs to know something about this rapidly growing field. Most simulations imply an optimization in the background—the simulation is being done to optimize some aspect of the situation.

A central idea of mathematics is *linear independence*. In computing it is

natural that this idea, which involves a yes no situation, becomes more vague. Clearly in computing there will be some degree of linear independence, and various forms for representing the same information will have varying degrees of linear independence. The idea is still in its infancy and needs a great deal more development, but it is clearly a central idea in computing.

One of the more difficult problems requiring an algorithm is finding the eigenvalues and eigenvectors of a matrix. Unfortunately, there is a great lack of understanding of what the problem actually is and of what the answers are to be used for, and there are no widely accepted methods for the general case. For the particular case of a symmetric (also for a Hermitian) matrix reasonably effective methods are known.

1.9 REFERENCES

The problem of supplying further references is a vexing one. The literature is rapidly changing when compared to the lifetime of a book, and as a result most references would soon be out of date and misleading. Furthermore, in a text like this where most chapters can, and some have, been expanded into whole books, there is little point in giving a lot of isolated references which will probably be ignored by most readers. We shall assume that a few standard textbooks are available and usually refer the reader to them for further information. The occasional reference to the literature is to amplify a point that is not in standard textbooks.

2

NUMBERS

2.1 INTRODUCTION

Numbers are the basis of numerical methods. Thus logically they belong at the beginning of a course on numerical methods. On the other hand, psychologically they occur rather late in the development.

The situation in numerical methods is very like that in the calculus course where the real number system is basic to the limit process. The calculus course, therefore, often begins with a detailed, fairly rigorous discussion of the real number system. Unfortunately, at this point in his education the student has little reason to care about the topic, and it always turns out to be the most difficult part of the entire course. Furthermore, the topic generally repels the student, and this attitude carries over to the rest of the course.

The history of mathematics further shows that the real number system was very late in developing. The discoverers and developers of the calculus ignored the niceties of the number system for many years. The biological principle "ontogeny recapitulates phylogeny" means that "the development of the individual tends to repeat the development of the species." This is very relevant to teaching; the history of a subject gives important clues as to the ordering and relative difficulties of the material being taught.

History likewise shows that for a long time the number system used in computing was essentially ignored and taken for granted. Putting the topic first, therefore, requires justification, because we are asking the beginner to learn material whose importance he is not psychologically prepared to accept. The justification is the same as that for the calculus course. If we are to make rapid progress and not to have to repeat some material several times, then it is necessary to start with a firm foundation. The author's own experience was that only after many years of computing did he come to understand how the number system used by the machines affected what was obtained and how at times it led him astray.

Thus we are asking the beginner to accept on faith that the material in this chapter is basic and to put aside his natural psychological prejudices in favor of the logical approach. Of course he wants to get on to solving real problems and not to fuss with apparently trivial, irrelevant details of the number system used by machines, which he thinks he understands anyway. In compensation we will try to make the material a bit more dramatic than usual in order to sustain his interest through this desert of logical presentation. Probably he should plan to review this chapter later several times until he becomes thoroughly familiar with many of the various peculiar features of the number system used by the machine. In a sense it is the *real* number system, since they are the only numbers that can occur in the computation; there are no other numbers, and the mathematician's "real" number system is purely fictitious.

2.2 THE THREE SYSTEMS OF NUMBERS

There are three systems of numbers in the usual computing machine. First there are the *counting numbers* 0, 1, 2, . . . , 32,767 (or some other finite, rather small number). Note that this system begins with 0 rather than 1. Unless this fact is thoroughly learned, when a loop is written in a program, the loop will not be able to be done *no* times (meaning that it cannot be skipped). This number system is usually intimately connected with the index registers of the machine, and the range of the numbers is thereby determined. This is the familiar counting system, and little more need be said beyond the fact that it is definitely bounded and does not extend to infinity.

The second system of numbers is the familiar *fixed-point number* system. Typical numbers are:

$$3.141592654$$
$$0.012345678$$
$$-123.4567890$$

These numbers all have a fixed length, usually the word length of the machine (or some simple multiple of it), and the differences between successive numbers are the

same. They are the familiar numbers of hand computing. Almost all the tables of numbers that the beginner has used (trigonometric, logarithmic, etc.) are in fixed-point notation. The chief difference between the machine's system and hand calculation is that the human often changes the number of digits he carries as circumstances seem to warrant, while the machine generally keeps the same number of digits (it may occasionally shift from single to double or even multiple precision).

The third system of numbers is the *floating-point number* system, which is closely related to the so-called "scientific notation" used in many parts of science. The system is designed to handle both very large and very small numbers. Typical numbers are:

$$.31415927 \times 10^{1}$$
$$.12345678 \times 10^{-1}$$
$$-\underbrace{.12345678}_{\text{mantissa}} \times 10^{4\,\nwarrow\,\text{exponent}}$$

The first block of eight digits in these above numbers is called the *mantissa* (in analogy with logarithms), and the last digit is called the *exponent* (often ranging from at least -38 to $+38$).

Usually the mantissa and the exponent are stored in the same word of the machine, and as a result the mantissa of a floating-point number is usually shorter than the corresponding fixed-point number.

For convenience we shall use in the examples in the book a three-digit mantissa and a one-digit exponent for our floating-point number system. Not only does this save space, but it also makes the examples much easier to follow. Occasionally we will use a three-digit fixed-point number system. Examples of our floating-point numbers are:

$$\pi = .314 \times 10^{1}$$
$$\frac{1}{\sqrt{2}} = .707 \times 10^{0}$$
$$\frac{-\pi}{1{,}000} = -.314 \times 10^{-2}$$
$$0 = .000 \times 10^{-9}$$

2.3 FLOATING-POINT NUMBERS

The floating-point number system has a number of unfamiliar properties, and the rest of this chapter is devoted to examining some of them.

First, the zero

$$0 = .000 \times 10^{-9}$$

is relatively isolated from the adjacent numbers since the next two positive numbers

$$.100 \times 10^{-9} \qquad \text{and} \qquad .101 \times 10^{-9}$$

are 10^{-12} apart. No other number has a mantissa beginning with a zero digit.

Second, each decade has exactly the same number of numbers, 900 in all, running

$$\text{from } .100 \times 10^{a} \text{ to } .999 \times 10^{a} \qquad a = -9, -8, \ldots, 0, \ldots, 9$$

Within a decade the numbers are equally spaced, but the spacing increases each decade. Thus the spacing is a fixed, arithmetic spacing for 900 numbers, followed by a "geometric spacing jump" and then another block of 900 arithmetically spaced numbers, etc.

The number 0 and the number -0 (namely, $-.000 \times 10^{-9}$) are logically the same, and some machines make them the same, but some do not. Usually there is no infinity corresponding to zero.

In mathematics there is a single, unique zero, while in computing there are two kinds of zeros. The first, as in mathematics, occurs in expressions like

$$a \cdot 0 = 0$$

and behaves like a proper zero. But the zero that occurs in expressions like

$$a - a = 0$$

can come from a form like

$$1 - (1 - \varepsilon)(1 + \varepsilon) = 0$$

whenever ε^2 is less than $\frac{1}{2}\, 10^{-3}$ (3 being the number of decimal places carried in the mantissa). Clearly this zero differs from the mathematical zero. It is this zero that arises in the subtraction of two apparently equal-sized numbers which causes so much trouble when the finite arithmetic of the machine is equated to the infinite arithmetic of mathematics.

All these remarks appear to be obvious, but their consequences continually affect how we compute and the results we get from the machine.

It is natural in a floating-point number system to measure the error of a number by the size of the difference *relative to* the correct number, and thus

$$\text{Relative error} = \left| \frac{\text{true} - \text{calculated}}{\text{true}} \right|$$

This is distinctly different from the conventional *absolute error* used in much of mathematics, where

$$\text{Absolute error} = |\text{true} - \text{calculated}|$$

The idea of relative error fits most physical situations very well, since it is *scale-free*; that is, a change in the size of the units of measurement does not change the size of the relative error, while it does change the absolute error.

The use of relative error fails, however, when the true value is zero, or close to it. For example, in computing $\sin \pi$ we would have

$$\text{Relative error} = \left| \frac{\sin \pi - \sin(.314 \times 10^1)}{\sin \pi} \right|$$

and since $\sin(.314 \times 10^1)$ is not likely to be exactly zero, the relative error would be infinite. For this reason it is often better in computing the relative error to use

$$\max\{|x|, |f(x)|\}$$

as the denominator rather than $f(x)$.

As a result of the discrete spacing of the numbers, there are numbers that cannot come out of some calculations. For example, consider evaluating $\tan x$. The slope of the function is always greater than 1; therefore as we go through adjacent numbers in x, the corresponding values of $\tan x$ (see Fig. 2.3.1) must

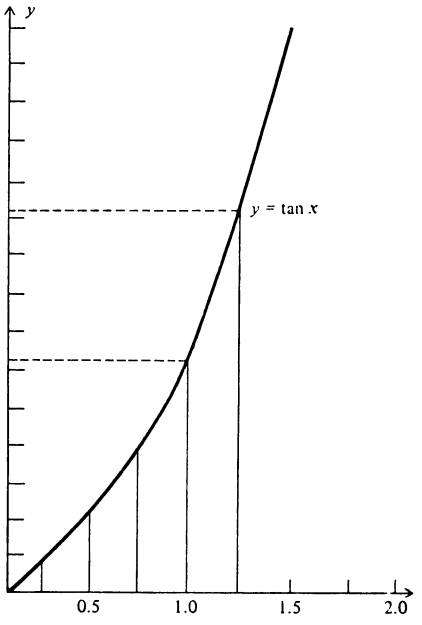


FIGURE 2.3.1
Graph of $y = \tan x$.

occasionally skip over some of the possible numbers (since both x and y start at the origin and y has gotten further in its range than has x in its range).

When searching for gaps in the function values for consecutive values of the argument, a somewhat crude rule to use is that the spacing is (roughly) proportional to the size of the number. This produces the spacing in the y values (roughly) proportional to $x(dy/dx)$, and if the spacing in the number system around y is finer, then there will be gaps, that is, if

$$|y| < \left| x \frac{dy}{dx} \right| \qquad \text{or} \qquad |y/x| < \left| \frac{dy}{dx} \right|$$

This can be translated into words as follows: when the slope of the line from the origin to the current point on the curve is less than the derivative at that point, there are likely to be gaps between consecutive function values.

It might be thought that if the slope is less than 1, as for the sine function in the first quadrant, then there would be no missed values, but this neglects the fine structure of the number system. Consider the table (in radians)

x	$\sin x$
1.00	.841
1.01	.847
1.02	.852
1.03	.857
1.04	.862

We see that x has shifted to a new decade and hence to a coarser spacing than that of the function values, thus producing the gaps in the sequence of function values.

PROBLEMS

2.3.1 Show that our number system has exactly 34,201 different numbers.

2.3.2 Discuss the gaps in the y values of $y = \sqrt{x}$.

2.3.3 Discuss the gaps in the y values of $y = x^2$.

2.3.4 Discuss the gaps in $y = \cos x$ for the first quadrant.

2.4 HOW NUMBERS COMBINE

Having looked at the individual numbers, let us now look very briefly at how numbers combine in the four arithmetic operations. We assume familiarity with conventional arithmetical practice.

The product of two three-digit numbers in conventional arithmetic is a five- or six-digit number, but in our number system there are only three-digit numbers! Which number shall we take as the product? Common sense suggests taking the three-digit number closest to the product. The following is a mechanism for doing this. If the leading digit of the full six-digit product is a zero, then shift the mantissa one position to the left and at the same time decrease the exponent (which is the sum of the exponents of the two factors) by 1 to compensate for the shift. Neglecting the sign of the product, add a 5 in the fourth position. Provided an overflow on the left does not occur, the first three digits of what remains are taken as the product. If an overflow does occur, then further shifting and adjusting of the exponent are required.

$$
\begin{array}{l}
\quad .512 \times 10^{1} \\
\times \ .106 \times 10^{2} \\
\hline
\quad 3072 \\
\quad 0000 \\
\quad 0512 \\
\hline
.054272 \times 10^{3} \\
.542720 \times 10^{2} \quad \text{shift left} \\
+ \ \ 5 \qquad\quad \text{round product} \\
\hline
.543 \quad \times 10^{2} = \text{product}
\end{array}
$$

This process of rounding produces a very slight *bias* because the ambiguous case of 500 in the last three places of the mantissa should be rounded up half the time and rounded down half the time, but the mechanism just described *always* rounds up. The effect is too slight to worry about in practice; it is chiefly of theoretical interest.

Division is somewhat more complicated, but the effect of rounding is much the same: we select the three-digit number closest to the mathematically correct quotient, again with a slight bias.

Addition and subtraction require first comparing the exponents of the two numbers and, if necessary, shifting one mantissa with respect to the other before combining. The final shifting for roundoff is much the same as for multiplication. Subtraction can produce many leading zeros (or even all zeros, in which case the machine produces $.000 \times 10^{-9}$), and these need to be shifted off before the rounding occurs. (Beware of using $.000 \times 10^{0}$ as zero.)

Compute $.314 \times 10^{1} + .419 \times 10^{-1}$.

$$\begin{array}{rll} & .314 \times 10^{1} & \\ + & .00419 \times 10^{1} & \\ \hline & .31819 \times 10^{1} & \\ + & 5 & \text{round} \\ \hline & .318 \times 10^{1} = & \text{sum} \end{array}$$

Compute $.315 \times 10^{2} - .314 \times 10^{2}$.

$$\begin{array}{rll} & .315 \times 10^{2} & \\ - & .314 \times 10^{2} & \\ \hline & .001 \times 10^{2} & \\ & .10000 \times 10^{0} & \text{shift left} \\ + & 5 & \text{round} \\ \hline & .100 \times 10^{0} = & \text{difference} \end{array}$$

Compute $.749 \times 10^{2} + .436 \times 10^{2}$.

$$\begin{array}{rll} & .749 \times 10^{2} & \\ + & .436 \times 10^{2} & \\ \hline & 1.185 \times 10^{2} & \\ & .1185 \times 10^{3} & \text{shift right} \\ + & 5 & \text{round} \\ \hline & .119 \times 10^{3} = & \text{sum} \end{array}$$

This briefly describes what ideally happens in roundoff. In practice there are many minor variations and some not so minor. For example, since this process of rounding is expensive in both hardware and machine speed, some machines merely *chop off* or drop the last digits once the initial shifting has lined up the leading digits. At first glance chopping may seem to be only moderately more severe than rounding, but unfortunately in some problems chopping can produce an "epidemic" in the sense that in cycle after cycle of computing the effect will be in the same direction and the cumulative effect will grow almost linearly. (See Sec. 23.4.)

PROBLEMS

2.4.1 Show that a shift due to roundoff can occur.

2.4.2 Discuss using $.000 \times 10^{e}$ as zero.

2.5 THE RELATIONSHIP TO MATHEMATICS AND STATISTICS

By now it should be clear that the mathematician's number system is significantly different from the floating-point number system of the machine, and the latter one is used in most scientific and engineering calculations. What relationship have they to each other?

Up to now we have adopted the harsh view that the numbers in the machine are the only numbers there are and that the mathematician's numbers are purely artificial. This is very useful for many purposes, especially when trying to debug a program by accounting for every last digit. It is also necessary if we are to understand the limitations of what can be done on the machine.

On the other hand, usually the machine is used to simulate the mathematician's system, and the machine's number system is sometimes a poor approximation of what is needed. The approximations in the initial numbers plus the continual roundoffs in almost every arithmetic operation produce differences that can be serious. This is especially true when a subtraction produces a large number of leading zeros which are then shifted off (and the exponent is correspondingly adjusted).

To understand roundoff *in the large*, it is customary to regard it as a *random process* in spite of the obvious fact that the same program run repeatedly will produce the same—not random—results (assuming that the machine is not defective). This assumption of random behavior is not essentially different from what is done in many real-world applications of statistics. In the statistical mechanics of gas molecules we simply do not want to know the detailed behavior of the 6.02×10^{23} molecules in a mole of gas; we merely want the effects of the average behavior. Similarly in practice we do not want to know the exact roundoff at all stages of a computation, rather we want to know their average effect.

Thus statistics continually enters into numerical methods because of the random roundoff effects. Furthermore, we usually deal only with samples of the functions and not with the functions themselves. As a result, statistics lies in the background of much of what we do, and occasionally it steps up into the foreground. It is necessary, therefore, whether we like the topic or not, to give serious attention to the statistical effects in computing.

2.6 THE STATISTICS OF ROUNDOFF

Let us look first at the distribution of the roundoff of a single number. In the mathematician's number system all numbers in a short interval occur with equal frequency. All numbers in the interval $x_0 - \frac{1}{2} \leq x_0 < x_0 + \frac{1}{2}$ (measured in units

of the last digit) go into the number x_0 (where $x_0 > 0$ is a number in the machine). Thus the roundoff has a uniform probability distribution in the last digit (Fig. 2.6.1).

$$p(x) = \begin{cases} 1 & (x_0 - \frac{1}{2}, x_0 + \frac{1}{2}) \\ 0 & \text{otherwise} \end{cases}$$

$$\int_{-\infty}^{\infty} p(x)\,dx = 1$$

The two most commonly used measures of a distribution are the mean (average) and the variance. The mean is the *first moment* of the distribution $p(x)$,

$$\text{Av}\{p(x)\} = \int_{x_0-1/2}^{x_0+1/2} xp(x)\,dx = \int_{x_0-1/2}^{x_0+1/2} x\,dx = \frac{(x_0 - \frac{1}{2})^2 - (x_0 - \frac{1}{2})^2}{2}$$
$$= x_0$$

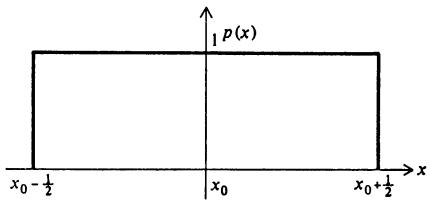


FIGURE 2.6.1
Distribution of roundoff.

while the variance is the *second moment about the mean* (measures the square of the "spread")

$$\text{Var}\{p(x)\} \equiv \sigma^2 = \int_{x_0-1/2}^{x_0+1/2} (x - x_0)^2 p(x)\,dx = \int_{-1/2}^{1/2} t^2\,dt = \tfrac{1}{12}$$

Experimental tests of the uniform distribution of roundoff seem to show that it is a reasonable model.

How shall we represent this roundoff? It is conventional to let x be the true (mathematician's) number and $x + \varepsilon$ be the computer number, where ε is the additive roundoff. This notation is suitable for the fixed-point number system, but for floating-point numbers it is better to use

$$x(1 + \varepsilon) \qquad |\varepsilon| \leq \tfrac{1}{2} \times 10^{-2} \qquad (2.6.1)$$

We have chosen to let x be the mathematician's number rather than the computer's number because in this book we are concerned with computing more from the user's point of view than from the machine's. The difference in notation be-

tween the two approaches is minor in principle, but it has the effect of focusing the attention on one aspect or another in *every* formula in which the roundoff occurs.

Roundoff, once started, propagates through subsequent operations. For example, in the multiplication of two numbers

$$x_1(1 + \varepsilon_1)x_2(1 + \varepsilon_2) = x_1x_2(1 + \varepsilon_1 + \varepsilon_2 + \varepsilon_1\varepsilon_2)$$

Usually $\varepsilon_1\varepsilon_2$ can be neglected, but we need to add the roundoff ε from the present operation to get the total roundoff

$$x_3(1 + \varepsilon_3) = x_1x_2(1 + \varepsilon_1 + \varepsilon_2 + \varepsilon) \qquad |\varepsilon| < \tfrac{1}{2} \times 10^{-2}$$
$$\varepsilon_3 = \varepsilon_1 + \varepsilon_2 + \varepsilon$$

Similarly for the other operations.

PROBLEMS

2.6.1 Calculate the mean and variance for chopping (see Sec. 2.4).
2.6.2 Examine the propagation of roundoff through division.
2.6.3 Show that for roundoff the third moment about the mean is 0 and the fourth is 1/80.

2.7 THE BINARY REPRESENTATION OF NUMBERS

The various features of the floating-point number system have been discussed in terms of the familiar decimal representation. However, most computing is done on machines that use the binary form for representing numbers. The binary representation system is increasingly familiar these days, and so only a few details will be given. As a general rule, if you have trouble with the binary system, then probably it is because you do not really understand the decimal system, and the way out of your trouble is to think about the similar situation in decimals. For example, a decimal number means

$$1{,}414.214 =$$
$$1 \times 10^3 + 4 \times 10^2 + 1 \times 10^1 + 4 \times 10^0 + 2 \times 10^{-1} + 1 \times 10^{-2} + 4 \times 10^{-3}$$

Similarly the binary number

$$1011.101 =$$
$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

It is important to recognize the difference between the *kind* of number system used (counting, fixed, or floating) and the *form* of the representation of the number

(binary, octal, decimal, hexadecimal, etc.). Regardless of the form of the representation, the number is the same; thus *3* in decimal and *11* in binary *are the same number.* It is conventional to speak of the binary number or the decimal number to save words, but in careful thinking it is necessary to differentiate between the number system and the form of the representation.

Perhaps the main stumbling block with the binary system is converting from decimal to binary and back. For example, consider writing the number 417 in the binary representation. That is, we wish to write 417 as a sum of powers of 2 with coefficients of either 0 or 1

$$417 = 1 \cdot 2^n + a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \cdots + a_0 2^0$$

If we divide both sides by 2, the remainder is a_0. If we divide the quotient by 2, the remainder is a_1, and so on.

$$\begin{array}{r|l} 2 & 417 \\ 2 & 208 + 1 = a_0 \\ 2 & 104 + 0 = a_1 \\ 2 & 52 + 0 = a_2 \\ 2 & 26 + 0 = a_3 \\ 2 & 13 + 0 = a_4 \\ 2 & 6 + 1 = a_5 \\ 2 & 3 + 0 = a_6 \\ 2 & 1 + 1 = a_7 \\ 2 & 0 + 1 = a_8 \end{array}$$

$$417 = 110 \quad 100 \quad 001$$

To convert back we reverse the process. Multiply a_8 by 2 and add a_7; multiply the sum by 2 and add a_6; etc.

$$\begin{array}{l} 1 \\ 2 \\ \overline{2 + 1 = 3} \\ \quad \times 2 \\ \quad \overline{6 + 0 = 6} \\ \quad\quad \times 2 \\ \quad\quad \overline{12 + 1 = 13} \\ \quad\quad\quad \times 2 \\ \quad\quad\quad \overline{26 + 0 = 26} \\ \quad\quad\quad\quad \text{etc.} \end{array}$$

The above process works for integers. For the fractional part we use a similar trick of doubling and using the overflow on the left. For example,

$$.762 = a_{-1}2^{-1} + a_{-2}2^{-2} + a_{-3}2^{-3} + \ldots$$

doubie

$$1.524 = a_{-1} + a_{-2}2^{-1} + a_{-3}2^{-2} + \ldots$$

and so $a_{-1} = 1$.

In full

$$\begin{array}{r|l} & .762 \\ & \underline{\quad 2} \\ 1 & 524 \\ & \underline{\quad 2} \\ 1 & 048 \\ & \underline{\quad 2} \\ 0 & 096 \\ & \underline{\quad 2} \\ 0 & 192 \\ & \underline{\quad 2} \\ 0 & 384 \\ & \underline{\quad 2} \\ 0 & 768 \\ & \underline{\quad 2} \\ 1 & 536 \end{array}$$

Hence

$$.762 = .110\,000\,1\ldots$$

Reversing the process gets from the binary representation to the decimal.

Previously prepared tables for conversion purposes are another way of converting from one form of representing a given number to another form of representing the same number.

The conversions are customarily done by the computing machine, and since the machine works in binary arithmetic and since the above discussion has used decimal arithmetic, there are differences in exactly what happens in the machine. It is not hard to take the machine's point of view and to deduce how to convert its way.

Some words of caution are needed however. The terminating decimal

$$.1 = \tfrac{1}{10} = .0001\ 1001\ 1001\ 1\ldots$$

does not terminate in binary, and as a result adding 1/10 to itself 10 times will not produce exactly 1 (just as in conventional decimals $1/3 + 1/3 + 1/3 = .333 + .333 + .333 = .999 \neq 1$). Thus using the floating-point representation of numbers for counting or for logical control is inviting trouble.

Another point of warning is needed. The conversion routines cannot always read in a decimal number, convert it to binary and back, and give the result that is the same as the original number—at least not when the number of digits in the two forms is reasonably balanced. Take, for example, a three decimal to ten binary digit (binary digit is usually abbreviated *bit*) conversion. The 100 decimal numbers

$$\left.\begin{array}{l} 9.00 \rightarrow 1001.000000 \\ 9.01 \rightarrow 1001.xxxxxx \\ 9.02 \rightarrow 1001.xxxxxx \\ \cdots\cdots\cdots\cdots\cdots \\ 9.99 \rightarrow 1001.\underbrace{xxxxxx}_{\text{64 distinct numbers}} \end{array}\right\} \text{100 numbers}$$

must go into 64 binary representations, so that some distinct decimals *must go into the same* binary representation and cannot be distinguished in the reconversion process. Thus in spite of the fact that $10^3 < 2^{10} = 1{,}024$, there will be times when a decimal number is read in and comes back slightly, and annoyingly, different.

PROBLEMS

Convert to binary:

2.7.1 1,728
2.7.2 1,972
2.7.3 1,066
2.7.4 0,345
2.7.5 0.592
2.7.6 1/12
2.7.7 1/16

Convert to decimal:

2.7.8 101 001
2.7.9 111 111

2.7.10 100 001
2.7.11 .111 111
2.7.12 .100 001
2.7.13 Show that the above argument for the nonunique conversion also applies to the conversion from 8 decimals to 27 binary digits.
2.7.14 Describe the terminating decimals that also terminate in binary.

2.8 THE FREQUENCY DISTRIBUTION OF MANTISSAS

Although the mantissas of floating-point numbers are equally spaced, and hence occur equally frequently in the form of representation, *they are not equally frequent in practice.* Instead, the probability of getting the leading digit 1, 2, or 3 in a decimal number is about 60 percent. For example, consider the 50 physical constants whose leading digits are given in Table 2.8.1. In Sec. 2.9 we shall show why we care about this phenomenon beyond mere curiosity.

This effect can be explained in terms of the mathematician's smooth number system since it is a characteristic of the numbers themselves and not peculiar to the finite representation that the machine necessarily uses. We shall show that it is

Table 2.8.1 THE DISTRIBUTION OF THE LEADING DIGITS OF 50 PHYSICAL CONSTANTS*

Leading digit N	Number of cases observed	Theoretical number Eq. (2.8.3)	Difference
1	16	15	1
2	11	9	2
3	2	6	−4
4	5	5	0
5	6	4	2
6	4	3	1
7	2	3	−1
8	1	3	−2
9	3	2	1
	50	50	

*From Handbook of Mathematical Functions, *AMS 55*, National Bureau of Standards, 1964 and Dover Publications, Inc.

reasonable to adopt the model for the distribution that the *probability density* for observing the number x in the base b is

$$r(x) = \frac{1}{x \ln b} \qquad \frac{1}{b} \leq x < 1 \tag{2.8.1}$$

This is called the *reciprocal distribution* (Fig. 2.8.1) for obvious reasons.

The cumulative probability distribution is defined as

$$R(x) = \int_{1/b}^{x} r(t)\, dt = \frac{\ln x + \ln b}{\ln b}$$

$$R\left(\frac{1}{b}\right) = 0 \qquad \text{and} \qquad R(1) = 1 \tag{2.8.2}$$

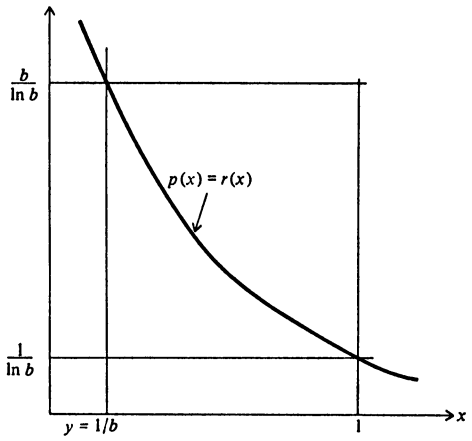


FIGURE 2.8.1
Reciprocal distribution.

Thus the probability of observing the leading digit N is

$$R\left(\frac{N+1}{b}\right) - R\left(\frac{N}{b}\right) = \frac{\ln (N+1) - \ln N}{\ln b} \tag{2.8.3}$$

which is what the table confirms.

We examine first the way multiplication transforms various distributions. Let x come from the probability (density) distribution $f(x)$, let y come from $g(y)$, and let the product z have the distribution $h(z)$. Further, let their cumulative distributions be, respectively, $F(x)$, $G(y)$, and $H(z)$, the measure of the set of points for which $xy \leq z$.

A study of Fig. 2.8.2 shows that

$$H(z) = \int_{1/b}^{z} \int_{1/b}^{z/(bx)} f(x)g(y)dy\,dx + \int_{1/b}^{z} \int_{1/(bx)}^{1} f(x)g(y)\,dy\,dx + \int_{z}^{1} \int_{1/(bx)}^{z/x} f(x)g(y)\,dy\,dx$$

$$= \int_{1/b}^{z} f(x)\left[G\left(\frac{z}{bx}\right) - G\left(\frac{1}{b}\right) + G(1) - G\left(\frac{1}{bx}\right)\right] dx + \int_{z}^{1} f(x)\left[G\left(\frac{z}{x}\right) - G\left(\frac{1}{bx}\right)\right] dx$$

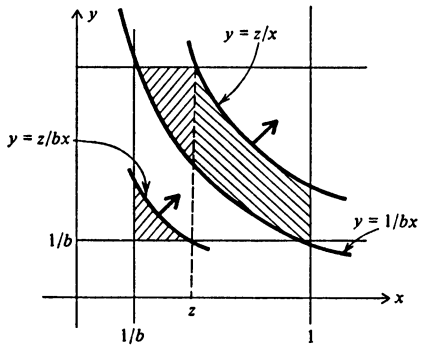


FIGURE 2.8.2
The cumulative probability distribution for the product $z = xy$.

Differentiating $H(z)$ with respect to z gives the density distribution

$$h(z) = f(z)\left[G\left(\frac{1}{b}\right) - G\left(\frac{1}{b}\right) + G(1) - G\left(\frac{1}{bz}\right) - G(1) + G\left(\frac{1}{bz}\right)\right] + \int_{1/b}^{z} f(x)g\left(\frac{z}{bx}\right)\frac{1}{bx}\,dx + \int_{z}^{1} f(x)g\left(\frac{z}{x}\right)\frac{1}{x}\,dx$$

$$= \frac{1}{b}\int_{1/b}^{z} \frac{f(x)}{x} g\left(\frac{z}{bx}\right) dx + \int_{z}^{1} \frac{f(x)}{x} g\left(\frac{z}{x}\right) dx \qquad (2.8.4)$$

This is the basic formula which describes how the process of multiplication combines the distributions of mantissas of two numbers x and y to give the distribution $h(z)$ of the mantissa of the product.

Suppose, now, that one of the two factors, say y, has the reciprocal distribution; that is,

$$g(y) = \frac{1}{y \ln b}$$

Putting this in Eq. (2.8.4), we get

$$h(z) = \frac{1}{b}\int_{1/b}^{z} \frac{f(x)}{x}\frac{bx}{z\ln b}\,dx + \int_{z}^{1} \frac{f(x)}{x}\frac{x}{z\ln b}\,dx$$
$$= \frac{1}{z\ln b}\left(\int_{1/b}^{z} f(x)\,dx + \int_{z}^{1} f(x)\,dx\right) = \frac{1}{z\ln b} \qquad (2.8.5)$$

Obviously the same applies if we assume $f(x)$ is the reciprocal distribution. Thus we have shown that if one of the factors of a product comes from the reciprocal distribution, then regardless of the distribution of the other factor, the product has the reciprocal distribution. This may be called the *persistence of the reciprocal distribution* once it is established. In a long sequence of multiplications if at least *one* factor has the reciprocal distribution, then the result has the reciprocal distribution.

Next we show how the reciprocal distribution can arise. We need a measure of how close a distribution is to the reciprocal distribution. After some tries we measure the distance of $h(z)$ from the reciprocal distribution $r(z)$ by

$$\max_{1/b\le z\le 1}\left\{\left|\frac{h(z)-r(z)}{r(z)}\right|\right\} \equiv D\{h(z)\} = D\{h\} \qquad (2.8.6$$

This measures the maximum relative error (which is natural for floating-point numbers).

We just showed that

$$r(z) = \frac{1}{b}\int_{1/b}^{z} \frac{f(x)}{x} r\left(\frac{z}{bx}\right)dx + \int_{z}^{1} \frac{f(x)}{x} r\left(\frac{z}{x}\right)dx$$

Subtract this from Eq. (2.8.4) and divide by $r(z)$ to get the form for the distance function

$$\frac{h(z)-r(z)}{r(z)} = \frac{1}{b}\int_{1/b}^{z} \frac{f(x)}{x}\left\{\frac{g[z/(bx)]-r[z/(bx)]}{r(z)}\right\}dx$$
$$+ \int_{z}^{1} \frac{f(x)}{x}\left[\frac{g(z/x)-r(z/x)}{r(z)}\right]dx$$

But

$$bxr(z) = \frac{bx}{z\ln b} = r\left(\frac{z}{bx}\right)$$
$$xr(z) = \frac{x}{z\ln b} = r\left(\frac{z}{x}\right)$$

and we have

$$\frac{h(z)-r(z)}{r(z)} = \int_{1/b}^{z} f(x)\left\{\frac{g[z/(bx)]-r[z/(bx)]}{r[z/(bx)]}\right\} dx + \int_{z}^{1} f(x)\left[\frac{g(z/x)-r(z/x)}{r(z/x)}\right] dx$$

Since $f(x) \geq 0$ in the two intervals

$$\left|\frac{h(z)-r(z)}{r(z)}\right| \leq \int_{1/b}^{z} f(x)D\{g\}\,dx + \int_{z}^{1} f(x)D\{g\}\,dx \leq D\{g\}$$

for all z, it follows that

$$D\{h\} \leq D\{g\} \qquad (2.8.7)$$

How fast does the distribution approach the limiting distribution? Remembering that for any distribution $g(x)$

$$\int_{1/b}^{1} [g(x)-r(x)]\,dx = 0$$

we form a guess at the increase that must occur in replacing the square brackets by their maximum $D\{g\}$. We can also calculate how the distance decreases for a continued product of a sequence of numbers taken from a flat (uniform) distribution. See Table 2.8.2

Similar results can be obtained for division. For example, Eq. (2.8.4) becomes

$$h(z) = \frac{1}{z^2}\int_{1/b}^{z} xf(x)g\left(\frac{x}{z}\right) dx + \frac{1}{bz^2}\int_{z}^{1} xf(x)g\left(\frac{x}{bz}\right) dx$$

and the rest follows to produce the corresponding results for division.[1]

Table 2.8.2 DISTANCE FROM THE FLAT DISTRIBUTION TO THE LIMITING DISTRIBUTION

Number of factors	Distance	Percentage of original distance
1	1.558	100.0
2	0.3454	22.2
3	0.0980	6.29
4	0.0289	1.85

[1] For further results see R. W. Hamming, On the Distribution of Numbers, *Bell Systems Technical Journal*, vol. 49, no. 8, pp. 1609–1625, October, 1970.

PROBLEMS

2.8.1 Derive the corresponding results for division.

2.8.2 If $f(x)=f(y)=b/(b-1)$, show that for multiplication

$$h(z)=\frac{b}{(b-1)^2}\left[\ln b-(b-1)\ln z\right]$$

2.8.3 If $f(x)=g(y)=b/(b-1)$, show that for division

$$h(z)=\frac{1}{2(b-1)}\left(b+\frac{1}{z^2}\right)$$

2.9 THE IMPORTANCE OF THE RECIPROCAL DISTRIBUTION

The reciprocal distribution has many applications. For example, consider the placing of the decimal or binary point before rather than after the first nonzero digit. Scientific convention places it after, while computing convention places it before, the digit. The difference first arose in the earliest electronic computers where the choice in fixed-point notation meant that the product would not produce an overflow on the left. What justification can we *now* give? If it is placed before, we run the risk in floating point of having a leading zero and of requiring time to shift this off and adjust the exponent accordingly. On the other hand if it is after the digit, we run the risk of getting two digits before the point and of requiring an exponent adjustment to get to standard form. What are the probabilities of these two (complementary) events? If $xy \leq 1/b$, then the probability of a shift when both factors are from the reciprocal distribution is (Fig. 2.9.1)

$$\begin{aligned}
p &= \int_{1/b}^{1}\int_{1/b}^{1/(bx)} \frac{1}{x\ln b}\frac{1}{y\ln b}\,dy\,dx \\
&= \int_{1/b}^{1}\frac{1}{\ln^2 b}\left[\frac{\ln 1/(bx)-\ln 1/b}{x}\right]dx = \int_{1/b}^{1}\frac{1}{\ln^2 b}\left(-\frac{\ln x}{x}\right)dx \\
&= \frac{1}{\ln^2 b}\left(-\frac{\ln^2 x}{2}\right)\Bigg]_{1/b}^{1} = \frac{1}{2}
\end{aligned}$$

And so it is a matter of indifference where the point is placed in this model. For numbers from the flat uniform distribution the probability depends on the base b and for $b=2, p\approx 0.38$.

In the design of optimal library routines it is necessary to know the distribution of the input data. In the cases of square root and exponential routines it is the distribution of the mantissa that is most important, but for other routines such as the sine it is necessary to know something about the distribution of the ex-

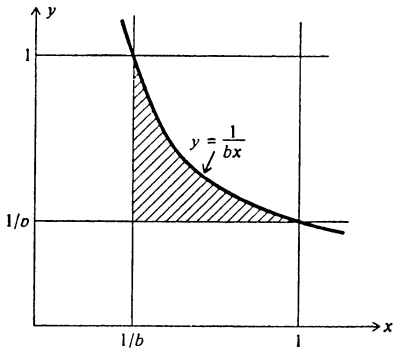


FIGURE 2.9.1
Probability of a shift.

ponents. Unfortunately almost nothing at this time is actually known, nor are there any theories available to suggest what might be found under some suitable conditions.

We will later give some other applications of the reciprocal distribution.

PROBLEMS

2.9.1 In a base 16 system written as groups of four binary digits, show that the numbers of the form

.1xxx ...
.01xx ...
.001x ...
.0001 ...

are equilikely for the reciprocal distribution.

2.9.2 In information theory the information measure of a distribution is

$$I = -\int_{-\infty}^{\infty} p(x)\ln[p(x)]\,dx$$

Apply this to the reciprocal distribution and compute the information loss from the uniform distribution for base b.

2.10 HAND CALCULATION

Hand calculations are used to see how a computation might go *before* programming it for a machine and to check the results after a machine run. We no longer need to do extensive hand calculations, and as a result it is a dying art—one that is

best left to die quietly. A whole book could be devoted to the topic, and the knowledge of its contents would be of some use sometimes, but it is simply not worth the effort to learn in detail when there are so many other things more worth knowing. Besides, the topic is dull and disorganized.

The main thing to note is the types of error that are most common in hand calculation. The two most prominent errors are both in copying numbers. The first is to reverse a pair of digits—87 becomes 78—and the second is to double the wrong number in a triple of numbers—776 becomes 766. They are the same errors that are common in dialing phone numbers.

3

FUNCTION EVALUATION

3.1 INTRODUCTION

Once we understand the number system, we can examine how numbers combine when we evaluate functions. Mathematically equivalent formulas can have very different roundoff characteristics. If the way we evaluate a function produces a great deal of roundoff, then we cannot use it for practical computing. We need, therefore, to see how to *avoid* roundoff when evaluating functions; we need to learn how to evaluate functions for machine computation.

3.2 THE EXAMPLE OF THE QUADRATIC EQUATION

The formula for finding the zeros of the quadratic equation

$$ax^2 + bx + c = 0$$

is given in textbooks as

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

An examination of the formula suggests that when $4ac$ is small with respect to b^2, that is,

$$|4ac| \ll b^2$$

then

$$\begin{aligned} -b + \sqrt{b^2 - 4ac} \quad & \text{for } b > 0 \\ -b - \sqrt{b^2 - 4ac} \quad & \text{for } b < 0 \end{aligned}$$

will result in severe cancellation for one root. Consider, for example, the particular case:

$$x^2 - 80x + 1 = 0 \qquad \begin{cases} a = & 0.100 \times 10^1 \\ b = & -0.800 \times 10^2 \\ c = & 0.100 \times 10^1 \end{cases}$$

$$x = \frac{0.800 \times 10^2 \pm \sqrt{0.640 \times 10^4 - 0.400 \times 10^1}}{0.200 \times 10^1}$$

$$= \frac{0.800 \times 10^2 \pm 0.800 \times 10^2}{0.200 \times 10'}$$

Approximate true values

$$= \begin{cases} 0.800 \times 10^2 = 80 & 80 - \frac{1}{80} = 80\left(1 - \dfrac{1}{80^2}\right) \\ 0.000 \times 10^{-9} = 0 & \frac{1}{80} + \dfrac{1}{80^3} = \frac{1}{80}\left(1 + \dfrac{1}{80^2}\right) \end{cases}$$

How can we avoid this cancellation?
Since

$$a(x - x_1)(x - x_2) = ax^2 - a(x_1 + x_2)x + ax_1x_2$$

it follows that the product of the two zeros is c/a. If we use the case without the cancellation to find x_1 and then find x_2 from

$$x_2 = \frac{c}{ax_1}$$

we will get

$$x_1 = 0.800 \times 10^2$$

$$x_2 = \frac{1}{0.800 \times 10^2} = 0.125 \times 10^{-1}$$

We need, therefore, to rearrange the formula for the zeros to pick out the one without the cancellation. One way is

$$x_1 = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \qquad \text{Use opposite sign of } b$$

$$x_2 = \frac{c}{ax_1}$$

How good are these answers? One way of answering is to try to reconstruct the polynomial from the zeros.

$$(x - 0.800 \times 10^2)(x - 0.125 \times 10^{-1}) = x^2 - x(0.800 \times 10^2) + 0.100 \times 10^1$$

The reconstruction is exact; therefore, in some sense, the answers must be exact since we appear to have lost no information around the whole loop.

This is not the complete answer on how to evaluate the formula; we still need to worry about (1) underflow, (2) overflow, and (3) $b^2 - 4ac < 0$, but these are not relevant here.

PROBLEM

3.2.1 Find the formula for the roots of $ax^2 + 2bx + c = 0$ and note the savings in arithmetic.

3.3 REARRANGEMENT OF FORMULAS

It would appear that there are an unlimited number of tricks for rearranging formulas to avoid severe cancellation (in some region of the argument x), but they are often the *same* tricks that were used in the calculus course to rearrange the expressions that arose in the delta process of formally taking the derivative of a function.

EXAMPLE 3.1 Evaluate $\sqrt{x+1} - \sqrt{x}$ for large x. As in the calculus we rationalize the numerator by

$$(\sqrt{x+1} - \sqrt{x})\left(\frac{\sqrt{x+1} + \sqrt{x}}{\sqrt{x+1} + \sqrt{x}}\right) = \frac{1}{\sqrt{x+1} + \sqrt{x}}$$

which has no cancellation. ////

EXAMPLE 3.2 Evaluate $\dfrac{\sin x}{x}$ for small x.

In floating point there is no trouble except for $x = 0$. As a special case, if $x = 10^{-2}$, then

$$\sin x = 10^{-2}$$

and

$$\frac{\sin x}{x} = 1$$

as it should. In general since for small x

$$\sin x = x - \frac{x^3}{6} + \cdots$$

then

$$\frac{\sin x}{x} = 1 - \frac{x^2}{6} + \frac{x^4}{120} - \cdots$$

is accurately evaluated by the $\sin x$ routine followed by division by x. The trouble occurs in the fixed-point number system (which we do not use). ////

EXAMPLE 3.3 Evaluate $\sin(x + \varepsilon) - \sin x$ for small ε.
Using the trigonometric identity

$$\sin a - \sin b = 2 \cos \frac{a+b}{2} \sin \frac{a-b}{2}$$

we have

$$= 2 \cos \left(x + \frac{\varepsilon}{2}\right) \sin \frac{\varepsilon}{2}$$

as a suitable form. ////

EXAMPLE 3.4 Evaluate $\dfrac{1 - \cos x}{\sin x}$ for small x.

We have the identities

$$\frac{1 - \cos x}{\sin x} = \frac{\sin x}{1 + \cos x} = \tan \frac{x}{2}$$ ////

EXAMPLE 3.5 Evaluate

$$\int_N^{N+1} \frac{dx}{1 + x^2} = \arctan(N + 1) - \arctan N$$

for large N.

In Fig. 3.3.1 for large N the area is the shaded region and is expressed as the difference between two large areas. We use the identity

$$\arctan a - \arctan b = \arctan \frac{a-b}{1+ab}$$

to get

$$\arctan \frac{1}{1+N(N+1)}$$

as a suitable form. Note we also have avoided one arctan evaluation which is the time-consuming part of the computation. ////

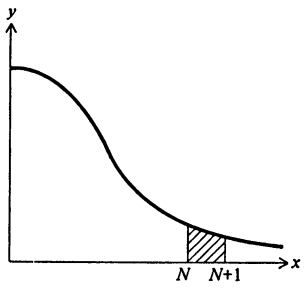


FIGURE 3.3.1

EXAMPLE 3.6 For large x evaluate

$$\frac{1}{\sqrt{x}} - \frac{1}{\sqrt{x+1}}$$

We write it as

$$\frac{\sqrt{x+1}-\sqrt{x}}{(\sqrt{x+1})(\sqrt{x})} = \frac{1}{\sqrt{x+1}\sqrt{x}(\sqrt{x+1}+\sqrt{x})} = \frac{1}{(x+1)\sqrt{x}+x\sqrt{x+1}} \quad ////$$

PROBLEMS

For large x, or ε small with respect to x, rearrange for evaluation:

3.3.1 $\dfrac{1}{x+1} - \dfrac{1}{x}$

3.3.2 $\tan(x+\varepsilon) - \tan x$ *Ans.* $\dfrac{\sin \varepsilon}{\cos x \cos(x+\varepsilon)}$

3.3.3 $\dfrac{1}{x+1}-\dfrac{2}{x}+\dfrac{1}{x-1}$ *Ans.* $\dfrac{2}{x(x^2-1)}$

3.3.4 $\sqrt[3]{x+1}-\sqrt[3]{x}$

3.3.5 $\cos(x+\varepsilon)-\cos x$

3.3.6 $\displaystyle\int_N^{N+1}\frac{dx}{x}=\ln(N+1)-\ln N$ N large

3.3.7 $e^{\varepsilon}-2+e^{-\varepsilon}$ *Ans.* $4\sinh^2(\varepsilon/2)$

3.4 SERIES EXPANSIONS

Sometimes rearrangements cannot be found to remove the cancellation, and some other device must be used. One of the more effective tricks from the calculus course is the expansion of the functions into a Taylor series about some suitable point.

EXAMPLE 3.7 Evaluate e^x-1 for small x.
Expanding e^x about $x=0$, we get

$$\begin{aligned} e^x-1 &= 1+x+\frac{x^2}{2}+\frac{x^3}{6}+\cdots-1 \\ &= x\left(1+\frac{x}{2}+\frac{x^2}{6}+\cdots\right) \end{aligned}$$ ////

EXAMPLE 3.8 Evaluate for large N

$$\begin{aligned} \int_N^{N+1}\ln x\,dx &= (x\ln x-x)\Big|_N^{N+1} \\ &= (N+1)\ln(N+1)-N\ln N-1 \end{aligned}$$

which has severe roundoff trouble in this form. See Fig. 3.4.1.

There are many ways of going about this problem. One way is to write it as

$$N[\ln(N+1)-\ln N]+\ln(N+1)-1=N\ln\left(1+\frac{1}{N}\right)+\ln(N+1)-1$$

and use

$$\ln(1+x)=x-\frac{x^2}{2}+\frac{x^3}{3}-\frac{x^4}{4}+\cdots \qquad |x| \text{ small}$$

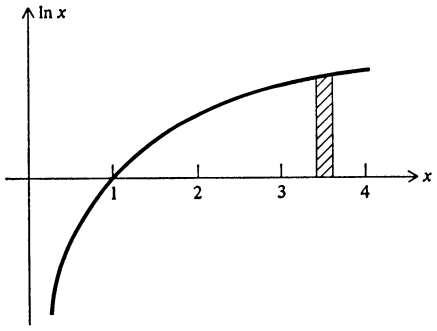


FIGURE 3.4.1

to get

$$= N\left(\frac{1}{N} - \frac{1}{2N^2} + \frac{1}{3N^3} - \frac{1}{4N^4} + \cdots\right) + \ln(N+1) - 1$$

$$= \ln(N+1) - \frac{1}{2N} + \frac{1}{3N^2} - \frac{1}{4N^3} + \cdots \qquad ////$$

EXAMPLE 3.9 Evaluate $\frac{1}{x} - \operatorname{ctn} x$ for small x.

We have

$$\frac{1}{x} - \operatorname{ctn} x = \frac{1}{x} - \frac{\cos x}{\sin x}$$

$$= \frac{\sin x - x\cos x}{x \sin x}$$

$$= \frac{\left(x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots\right) - x\left(1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \cdots\right)}{x\left(x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots\right)}$$

$$= \frac{\frac{x^3}{3}\left(1 - \frac{x^2}{10} + \cdots\right)}{x^2\left(1 - \frac{x^2}{6} + \cdots\right)} = \frac{x}{3}\left(1 + \frac{x^2}{15} + \cdots\right) \qquad ////$$

EXAMPLE 3.10 For small x evaluate

$$\frac{\ln(1-x)}{\ln(1+x)} = \frac{-\left(x+\frac{x^2}{2}+\frac{x^3}{3}+\frac{x^4}{4}+\cdots\right)}{x-\frac{x^2}{2}+\frac{x^3}{3}-\frac{x^4}{4}+\cdots}$$

Divide out the two series formally to get

$$\frac{\ln(1-x)}{\ln(1+x)} = -\left(1+x+\frac{x^2}{2}+\frac{5x^3}{12}+\cdots\right) \qquad ////$$

If we are given an ε that is small, then when we evaluate

$$\ln(1+\varepsilon)$$

by computing first $1+\varepsilon$, we lose most of the accuracy in this first addition. For this reason many math package libraries include the function:

given ε, compute $\ln(1+\varepsilon)$ directly

PROBLEMS

For large x, or ε small with respect to x, compute:

3.4.1 $\dfrac{1-\cos\varepsilon}{\varepsilon^2}$

3.4.2 $\dfrac{\varepsilon(e^{(a+b)\varepsilon}-1)}{(e^{a\varepsilon}-1)(e^{b\varepsilon}-1)}$ *One ans.* $\dfrac{a+b}{ab}$

3.4.3 $\ln\dfrac{1-\varepsilon}{1+\varepsilon}$ *Ans.* $-2\left(\varepsilon+\dfrac{\varepsilon^3}{3}+\dfrac{\varepsilon^5}{5}+\cdots\right)$

3.4.4 $\sqrt{\dfrac{e^{2\varepsilon}-1}{e^{\varepsilon}-1}}$

3.4.5 $\dfrac{\varepsilon-\sin\varepsilon}{\varepsilon-\tan\varepsilon}$

3.4.6 $(x+1)^{1/n}-x^{1/n}$

3.5 USE OF MACHINE TO DECIDE

The beginner is inclined to believe that he must personally analyze each problem and program the machine accordingly. Only after a while does it occur to him that the machine can make the choices, provided the program is written properly.

For example, consider evaluating

$$e^{ax} - 1$$

for a range of x and a set of parameter values a. When $|ax|$ is small, we need to use the series approximation; otherwise we can use the direct evaluation and subtract. If we do not mind the loss of one bit in the direct evaluation, then we need only use the series for $|ax| < \frac{1}{10}$. (*Note*: $e^{0.1} = 1.105\ldots$)

To evaluate $e^{ax} - 1$, see Fig. 3.5.1.

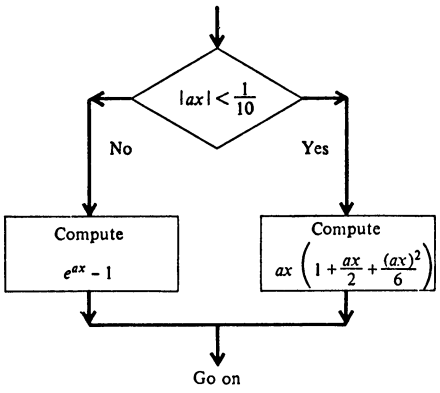


FIGURE 3.5.1

The term neglected is of relative size

$$\left|\frac{(ax)^3}{24}\right| < 10^{-3}$$

so that even if $ax < 0$, the relative error is less than $\frac{1}{2} \times 10^{-3}$, and there is no loss in accuracy in the power series approximation.

3.6 THE MEAN VALUE THEOREM

The mean value theorem is the third item from the calculus course that is widely used in computing, especially in the more theoretical parts of deriving errors of formulas.